

Prompt: Fechas y Filtros en Reservas/Check-in/Check-out + Simplificar Origen en Planning

PARTE 1 — Mostrar solo reservas activas/futuras por defecto + formato de fechas dd/mm/aaaa

Problema

- En "Reservas", "Check-in" y "Check-out" aparecen todas las reservas históricas mezcladas
- Las fechas se muestran en formato `yyyy-MM-dd` (ej: `2025-03-15`) en lugar de `dd/mm/aaaa` (ej: `15/03/2025`)

1a — Función global de formato de fecha

En `client/src/lib/utils.ts` (o al inicio de cada página donde se use), agregar:

```
export function formatDateAR(dateStr: string | null | undefined): string {
  if (!dateStr) return "-";
  // Parsear como fecha local evitando desfase de timezone
  const [year, month, day] = dateStr.split("-");
  return `${day}/${month}/${year}`;
}
```

1b — Aplicar en ReservationsPage

En `reservations.tsx`, importar `formatDateAR` y reemplazar:

```
// Buscar estas dos líneas en la tabla:
<TableCell>{reservation.checkInDate}</TableCell>
<TableCell>{reservation.checkOutDate}</TableCell>

// Reemplazar por:
<TableCell>{formatDateAR(reservation.checkInDate)}</TableCell>
<TableCell>{formatDateAR(reservation.checkOutDate)}</TableCell>
```

También agregar filtro por defecto para no mostrar reservas pasadas. En

filteredReservations , agregar condición:

```
const today = new Date().toLocaleDateString("en-CA", { timeZone: "America/Argenti

const filteredReservations = reservations?.filter((res) => {
  // Filtro de búsqueda y estado (existente)
  const guestName = `${res.guest?.firstName} ${res.guest?.lastName}`.toLowerCase(
  const matchesSearch =
    guestName.includes(searchQuery.toLowerCase()) ||
    res.room?.roomNumber?.toLowerCase().includes(searchQuery.toLowerCase()) ||
    res.reservationCode?.toLowerCase().includes(searchQuery.toLowerCase());
  const matchesStatus = statusFilter === "all" || res.status === statusFilter;

  // Ocultar reservas pasadas (checkout antes de hoy) a menos que el filtro sea "
  const isPast = res.checkOutDate < today && res.status === "checked_out";
  const isCancelledOld = res.checkOutDate < today && res.status === "cancelled";
  if (showHistory === false && (isPast || isCancelledOld)) return false;

  return matchesSearch && matchesStatus;
});
```

Agregar estado:

```
const [showHistory, setShowHistory] = useState(false);
```

Y botón toggle al lado del filtro de estado:

```
<Button
  variant={showHistory ? "default" : "outline"}
  size="sm"
  onClick={() => setShowHistory(!showHistory)}
  data-testid="button-toggle-history"
>
  <History className="h-4 w-4 mr-1" />
  {showHistory ? "Ocultar historial" : "Ver historial"}
</Button>
```

Agregar `History` a los imports de `lucide-react`.

1c — Aplicar formato de fechas en ReservationFormDialog (folio y detalle)

Buscar las líneas donde se muestran `checkInDate` y `checkOutDate` en el modal de

detalle de reserva (líneas cerca de `data-testid="text-checkin"` y `data-testid="text-checkout"`):

```
// Reemplazar:
<p className="font-semibold text-sm" data-testid="text-checkin">{reservation.chec
<p className="font-semibold text-sm" data-testid="text-checkout">{reservation.che

// Por:
<p className="font-semibold text-sm" data-testid="text-checkin">{formatDateAR(res
<p className="font-semibold text-sm" data-testid="text-checkout">{formatDateAR(re
```

También en el folio (línea con `checkInDate` } → {`selectedReservation.checkOutDate` }):

```
// Reemplazar:
<TableCell>{selectedReservation.checkInDate} → {selectedReservation.checkOutDate}
// Por:
<TableCell>{formatDateAR(selectedReservation.checkInDate)} → {formatDateAR(select
```

Y en la tabla de pagos del folio (línea con `p.date`):

```
// Reemplazar:
<TableCell className="text-sm">{new Date(p.date + "T12:00:00").toLocaleDateString
// Por:
<TableCell className="text-sm">{formatDateAR(p.date)}</TableCell>
```

1d — CheckInPage: mostrar solo de hoy en adelante

En `check-in.tsx`, la query usa `/api/reservations/check-in` que ya filtra por hoy y mañana — eso está bien. Pero también mostrar fechas en formato correcto en la lista. Buscar donde se renderiza `reservation.checkInDate` en la lista de check-in y aplicar `formatDateAR`.

1e — CheckOutPage: mostrar solo checked_in (sin pasadas)

La query ya filtra por `status === "checked_in"` — está bien. Solo aplicar `formatDateAR` donde se muestran las fechas.

PARTE 2 — Simplificar colores de origen en el Planning

Objetivo

Reducir de 11 colores/orígenes a solo 3 grupos visuales:

- **OTAs** (booking, expedia, airbnb, despegar, hotelbeds, agoda, ota) → un solo color
- **Directo** (directo, telefono, web) → un solo color
- **Empresa** (empresa, agencia) → un solo color

2a — Reemplazar `getSourceColor` en `planning.tsx`

Buscar la función `getSourceColor` y reemplazarla completamente:

```
function getSourceColor(source: ReservationSource): string {
  // OTAs - índigo
  if (["booking", "expedia", "airbnb", "despegar", "hotelbeds", "agoda", "ota"].i
    return "bg-indigo-200 dark:bg-indigo-800/60 border-indigo-300 dark:border-ind
  }
  // Directo - azul
  if (["directo", "telefono", "web"].includes(source)) {
    return "bg-blue-200 dark:bg-blue-800/60 border-blue-300 dark:border-blue-700"
  }
  // Empresa / Agencia - verde
  if (["empresa", "agencia"].includes(source)) {
    return "bg-emerald-200 dark:bg-emerald-800/60 border-emerald-300 dark:border-
  }
  return "bg-gray-200 dark:bg-gray-800/60 border-gray-300 dark:border-gray-700";
}
```

2b — Reemplazar `getSourceLabel` en `planning.tsx`

```
function getSourceLabel(source: ReservationSource): string {
  if (["booking", "expedia", "airbnb", "despegar", "hotelbeds", "agoda", "ota"].i
    return "OTA";
  }
  if (["directo", "telefono", "web"].includes(source)) {
    return "Directo";
  }
  if (["empresa", "agencia"].includes(source)) {
    return "Empresa";
  }
  return source;
}
```

2c — Reemplazar `sourceItems` en la leyenda del planning

Buscar el array `sourceItems` y reemplazarlo:

```
const sourceItems: { source: ReservationSource; label: string }[] = [
  { source: "directo", label: "Directo (tel / web / presencial)" },
  { source: "booking", label: "OTAs (Booking, Expedia, Airbnb, etc.)" },
  { source: "empresa", label: "Empresa / Agencia" },
];
```

Esto muestra solo 3 ítems en la leyenda con los colores correspondientes.

2d — Tooltip en cada celda del planning

En las celdas del planning donde se muestra el origen, el tooltip ya muestra el canal exacto (ej: “Booking”). Mantener ese comportamiento — el color agrupa visualmente pero al pasar el mouse se ve el canal real.

Si no hay tooltip, agregar `title` al div de la celda:

```
title={`${reservation.guestName} - ${getSourceLabel(reservation.source)} (${reser
```

2e — Aplicar mismo cambio en `sourceOptions` del formulario

En `QuickReservationDialog` y `ReservationFormDialog`, los selectores de “Canal/Origen” quedan igual (booking, expedia, etc. como opciones individuales) — el usuario sigue eligiendo el canal exacto al crear la reserva. Solo cambia cómo se muestra visualmente en el planning.

Resumen de archivos modificados

Archivo	Cambio
client/src/lib/utils.ts	Agregar función <code>formatDateAR</code>
client/src/pages/reservations.tsx	Filtro ocultar pasadas + botón historial + formato fechas
client/src/pages/check-in.tsx	Formato fechas en lista
client/src/pages/check-out.tsx	Formato fechas en lista y folio
client/src/pages/planning.tsx	<code>getSourceColor</code> , <code>getSourceLabel</code> , <code>sourceItems</code> simplificados

ReservationFormDialog **(component)**

Formato fechas en detalle y folio